Guía de integración frontend y diagnóstico del sistema

UTA SUMO ROBOTICS CUP 2026

Auditoría ejecutada: 22 de octubre de 2026

Rama inspeccionada: `feature/backend-core`

Alcance de cambios: backend, migraciones y documentación. El frontend se inspeccionó sin editarlo.

1. Resumen ejecutivo

El backend FastAPI, SQL Server y sus flujos principales existen y se verificaron contra la base local. La migración 22 agrega reglas configurables por evento/categoría y una regla por migración para Mini Sumo Amateur RC de 2026. El backend ya tiene registro/login por correo, Google ID token, equipos, robots, inscripción transaccional, pagos, comprobantes privados, correo con outbox, reglamentos versionados, reportes paginados, puntuación e integración WebSocket.

La interfaz todavía no está conectada de extremo a extremo. En particular, el formulario muestra tres puestos (dos participantes y coach), aunque el evento oficial requiere dos participantes y ningún coach. El panel administrativo de inscripciones y los resultados mantienen datos simulados en `localStorage`. No se encontraron vistas que consuman los reportes, la administración de eventos/categorías ni los reglamentos.

Las pruebas de backend ejecutadas al cerrar esta auditoría: 35 aprobadas con `RUN_SQLSERVER_TESTS=1`. La migración se aplicó a la base local existente sin reiniciarla. No se ejecutó `npm run build` ni se modificaron archivos del frontend.

2. Arquitectura actual observada

- Frontend: React 19 y Vite; servicios de API bajo `frontend/uta-sumo-web/src`.
- Backend: FastAPI, endpoints síncronos `def`, SQLAlchemy síncrono y pyodbc para SQL Server. El trabajo de endpoint síncrono es ejecutado por FastAPI en su thread pool; el acceso a base de datos no es async.
- Base de datos: SQL Server. Las migraciones numeradas se aplican en transacción por versión y se registran con checksum en `SchemaMigrations`.
- Autenticación: JWT Bearer; roles `ADMIN`, `EVENT_MANAGER` y `PARTICIPANT`; registro público solo asigna PARTICIPANT.
- Datos sensibles: cifrado de campos personales y referencias de pago, hashes de búsqueda, contraseñas Argon2id, PDFs privados cifrados y autorización de descargas.
- Correo: outbox persistente; el envío se hace desde el backend usando variables SMTP. El estado `SENT` indica aceptación del SMTP, no garantiza que el mensaje llegue a Inbox.
- Puntuación: movimientos auditables en SQL y marcador actual por WebSocket. El broadcast en memoria cubre un proceso; despliegues con varios workers requieren un broker compartido.

La API se publica bajo `/api/v1`; Swagger está disponible en `/docs` en el entorno local. El origen permitido del frontend se configura en `CORS_ORIGINS` del backend.

3. Trabajo backend completado en esta revisión

Reglas de roster configurables

Se eliminó la validación global que suponía una plantilla de dos participantes o dos más coach. El esquema admite de 1 a 30 integrantes y al menos un rol PARTICIPANT; la política específica se valida en servidor usando `EventCategories`.

La migración `database/scripts/22_event_category_rules.sql`:

- crea la relación evento/categoría y límites de participantes, coaches y cupo;
- conserva asociaciones existentes a partir de tarifas e inscripciones;
- amplía las posiciones persistidas de 3 a 30;
- no elimina ni reescribe inscripciones ni pagos anteriores.

La migración `database/scripts/23_official_event_category_rules.sql` fija para UTA SUMO ROBOTICS CUP 2026 / MINI SUMO AMATEUR RC exactamente 2 participantes, 0 coaches. Solo controla inscripciones nuevas; los datos históricos se conservan.

El endpoint administrativo para crear o editar las reglas es:

```
PUT /api/v1/admin/events/{event_id}/categories/{category_id}
Authorization: Bearer <token ADMIN>
Content-Type: application/json

{
  "is_active": true,
  "min_participants": 2,
  "max_participants": 2,
  "min_coaches": 0,
  "max_coaches": 0,
  "max_registrations": null
}
```

El frontend obtiene las reglas públicas con `GET /api/v1/events/{event_id}/categories`. Crear o activar una tarifa vincula la categoría al evento con límites generales por defecto. Configura después los límites reales antes de abrir las inscripciones.

4. Contratos que debe consumir el frontend

4.1 Configuración del cliente

En `frontend/uta-sumo-web/.env`:

```
VITE_API_BASE_URL=http://127.0.0.1:8000
VITE_GOOGLE_CLIENT_ID=<Client ID OAuth de tipo aplicación web>
```

`VITE_GOOGLE_CLIENT_ID` es un identificador público de aplicación web, no el secreto OAuth. Debe coincidir con `GOOGLE_CLIENT_ID` del backend y autorizar el origen del frontend en Google Cloud. El backend verifica el ID token recibido; no acepta contraseñas Google.

El frontend debe reiniciar Vite al cambiar sus variables. No se debe guardar en `VITE_*` ninguna contraseña de servicio, clave privada ni token SMTP.

4.2 Registro, login y sesión

Acción	Método y ruta	Resultado esperado
Registrar participante	`POST /api/v1/auth/register`	Usuario PARTICIPANT y token
Login manual	`POST /api/v1/auth/login`	Token y perfil
Login/registro Google	`POST /api/v1/auth/google`	Token tras verificar ID token
Vincular Google	`POST /api/v1/auth/google/link`	Vincula la identidad a sesión autenticada
Perfil autenticado	`GET /api/v1/auth/me`	Usuario y rol actuales
Cambiar contraseña	`POST /api/v1/auth/change-password`	Invalida tokens anteriores

Enviar `Authorization: Bearer <access_token>` en rutas protegidas. Manejar `401` como sesión vencida/no válida y `403` como permiso insuficiente. No inferir el rol desde el formulario; leer `user.role` de la respuesta autenticada.

Registro manual requiere email válido y contraseña de al menos 15 caracteres. Google es solo para participantes; cuentas privilegiadas se aprovisionan por operador backend. Los roles `REVIEWER` y `TREASURY` no son los roles actuales.

4.3 Formulario de inscripción

Al cargar:

1. Consultar `GET /api/v1/events?limit=100` y `GET /api/v1/institutions?limit=100`.
2. Para el evento seleccionado, consultar `GET /api/v1/events/{event_id}/registration-config` y `GET /api/v1/events/{event_id}/categories`.
3. Unir tarifas y reglas por `category_id`; renderizar el selector solo con categorías/tarifas disponibles.
4. Construir campos del roster desde `min_participants`, `max_participants`, `min_coaches` y `max_coaches`. Para el evento 2026 oficial son dos participantes y cero coaches. No enviar un coach fijo.
5. Crear equipo y robot propios con `POST /api/v1/teams` y `POST /api/v1/robots` si aún no existen.
6. Enviar `POST /api/v1/registrations` con `event_id`, `category_id`, `team_id`, `robot_id`, `participants` y `notes`.

Ejemplo del roster oficial:

```
{
  "event_id": 28,
  "category_id": 1,
  "team_id": 12,
  "robot_id": 9,
  "participants": [
    {"role":"PARTICIPANT","first_name":"Ana","last_name":"Pérez","institution_id":1,"mobile_
phone":"0990000001"},
    {"role":"PARTICIPANT","first_name":"Luis","last_name":"Mora","institution_id":1,"mobile_
phone":null}
  ],
  "notes": null
}
```

El backend determina la tarifa vigente. En una transacción crea la inscripción, el roster, un pago inicial pendiente, historial, PDF de solicitud de pago y registro de outbox. La respuesta incluye `amount`, `payment_id`, `receipt_url` y `email_delivery_status`. El PDF es una solicitud/instrucciones de pago, no una factura fiscal ni confirmación de pago.

4.4 Órdenes, comprobante y pagos

Necesidad	Método y ruta	Rol
Mis órdenes	`GET /api/v1/payments/my?page=1&limit=20`	PARTICIPANT
Mi inscripción	`GET /api/v1/registrations/my?page=1&limit=20`	PARTICIPANT
Descargar PDF de solicitud	`GET /api/v1/registrations/{id}/receipt`	Propietario o ADMIN
Cargar comprobante	`POST /api/v1/payments/{payment_id}/receipt` multipart `file`, opcional `payment_reference`	PARTICIPANT propietario
Descargar comprobante subido	`GET /api/v1/payments/{payment_id}/receipt`	Propietario o ADMIN
Revisar pago	`POST /api/v1/payments/{payment_id}/review`	ADMIN

Solo permitir subir si `can_upload_receipt` es `true`. Comprobantes permitidos: PDF, JPEG y PNG; el backend valida contenido, tamaño y firma y los almacena cifrados. No mandar manualmente `Content-Type` al usar `FormData`; el navegador debe añadir el boundary multipart. `IN_REVIEW` no significa pagado. Solo una decisión administrativa puede cambiar el pago a `PAID` o `REJECTED`.

4.5 Administración

- Login manual existente: `POST /api/v1/auth/login`; confirmar que `role` sea ADMIN o EVENT_MANAGER antes de abrir funciones administrativas.
- Usuarios y roles: `GET /api/v1/admin/roles`, `GET /api/v1/admin/users`, `PATCH /api/v1/admin/users/{user_id}`. Solo ADMIN gestiona cuentas.

- Catálogos: `GET/POST/PUT /api/v1/categories`, instituciones, eventos y tarifas según `/docs`. Configurar reglas por evento/categoría mediante el endpoint de la sección 3.
- Inscripciones administrativas: `GET /api/v1/admin/registrations` con filtros y paginación; historial por `GET /api/v1/registrations/{id}/history`; revisar registro con `POST /api/v1/registrations/{id}/review`.
- Pago: usar la ruta de revisión indicada arriba, solo con rol ADMIN.
- Inspeccionar/reintentar correo: `GET /api/v1/admin/email-outbox` y `POST /api/v1/admin/email-outbox/{outbox_id}/retry`; reenvío de documento `POST /api/v1/admin/registrations/{registration_id}/resend-receipt`.

El panel debe separar revisión de inscripción y revisión de pago: aprobar la inscripción no reemplaza verificar la transferencia.

4.6 Reportes y paginación

Vista	Endpoint
KPIs	`GET /api/v1/reports/overview?event_id={id}`
Inscripciones por estado	`GET /api/v1/reports/registrations/by-status?event_id={id}`
Inscripciones por categoría	`GET /api/v1/reports/registrations/by-category?event_id={id}`
Tendencia	`GET /api/v1/reports/registrations/trend?event_id={id}&from_date=YYYY-MM-DD&to_date=YYYY-MM-DD`
Resumen financiero	`GET /api/v1/reports/payments/summary?event_id={id}` solo ADMIN
Filas de inscripción	`GET /api/v1/reports/registrations/details?event_id={id}&page=1&limit=50`
Filas financieras	`GET /api/v1/reports/payments/details?event_id={id}&page=1&limit=50` solo ADMIN

Las filas detalladas se paginan en SQL. Leer `items`, `page`, `page_size`, `total` y `total_pages` del JSON; en listados generales también existen `X-Total-Count`, `X-Page`, `X-Page-Size`, `X-Total-Pages`. Construir filtros en servidor; no descargar todo para filtrar en el navegador. No mostrar importes financieros a EVENT_MANAGER.

4.7 Competencia y resultados

- Rondas: `GET/POST /api/v1/events/{event_id}/rounds`.
- Enfrentamientos: `GET/POST /api/v1/events/{event_id}/matches`.
- Estado: `PATCH /api/v1/events/{event_id}/competition-status`; consultar historial en `/competition-status/history`.
- Puntos: `POST /api/v1/events/{event_id}/scores`; movimientos e historial por `/scores/history` y `/scores/audit`.
- Tabla: `GET /api/v1/events/{event_id}/leaderboard`.
- Combate actual: `GET/PUT /api/v1/competition/combate-actual`; canal en vivo `ws://<api-host>/ws/combate-actual`.

Administrar rondas, enfrentamientos, estados y puntuación requiere ADMIN o EVENT_MANAGER. El leaderboard es público. La puntuación es append-only y requiere motivo; no mantener `localStorage` como fuente de verdad. En producción con varios procesos, el WebSocket necesita un broker compartido para propagar actualizaciones a todos los procesos.

4.8 Reglamentos

- Lista vigente pública: `GET /api/v1/events/{event_id}/regulations?category_id={id}`; si no hay reglamento específico, el backend puede devolver el general.
- Descargar: `GET /api/v1/regulations/{regulation_id}/file`.
- Administración: `GET/POST /api/v1/admin/events/{event_id}/regulations`, luego `POST /api/v1/admin/regulations/{id}/publish`.

Presentar título, versión/fecha y enlaces de descarga devueltos por el backend. Los archivos son PDF versionados en almacenamiento privado cifrado. La carga, publicación y archivo son funciones ADMIN.

5. Envío de correo y credencial SMTP

La clave de SMTP no es una clave que el frontend deba utilizar. El navegador no debe abrir conexiones SMTP ni recibir esa contraseña. La variable exacta del backend es:

```
SMTP_PASSWORD=<secreto SMTP del remitente>
```

Configurar su valor solamente en `backend/uta_sumo_api/.env` local (ignorado por Git) o en las variables privadas/secret manager del servidor backend. No copiar su valor a la guía, frontend, `VITE_*`, Postman ni al repositorio. El frontend consume `email_delivery_status` y `notification_status`; los valores usuales son `SENT`, `PENDING` y `FAILED`. Mostrar que la operación principal se guardó aunque el correo quede pendiente. `SENT` confirma aceptación SMTP, no entrega en Inbox.

Variables asociadas backend: `SMTP_HOST`, `SMTP_PORT`, `SMTP_USERNAME`, `SMTP_PASSWORD`, `SMTP_FROM_EMAIL`, `SMTP_FROM_NAME`, `SMTP_STARTTLS`. El Client ID Google y SMTP son credenciales distintas: Google autentica al participante; SMTP envía notificaciones desde el backend.

6. Hallazgos del frontend: tareas para sus desarrolladores

Prioridad	Evidencia del código actual	Trabajo que falta en frontend	Criterio de aceptación
Crítica	`ModuloIncripcion/components/Inscripcion.jsx` inicializa dos participantes y un coach; la regla oficial nueva permite solo dos participantes	Leer reglas del evento y generar exactamente los campos configurados	Crear inscripción 2026 con dos participantes funciona; un roster incompatible muestra el error backend sin crear datos parciales
Crítica	`ModuloAdministracion/services/adminInscripcionesApi.js` usa mocks y `localStorage`; `LoginAdmin.jsx` usa token simulado	Usar login real, roles, listado paginado y rutas de revisión	Una decisión actualiza SQL, recarga conserva resultado y PARTICIPANT recibe 403
Alta	`ModuloResultados/services/resultadosApi.js` documenta y persiste `localStorage`; existen APIs de puntuación y WebSocket	Conectar resultados a leaderboard, estado actual y WebSocket; borrar mock como fuente de verdad	Dos dispositivos reciben el marcador persistido y reflejan actualización en vivo
Alta	No se encontró una vista de reportes conectada a `/api/v1/reports`	Construir panel con filtros por evento, páginas SQL, gráficos/KPI y guardas de rol	Detalles cargan una página a la vez; EVENT_MANAGER no ve importes financieros
Alta	No se encontraron formularios administrativos de evento/categoría/reglamento consumiendo las nuevas rutas	Implementar catálogo por evento, límites, tarifas y publicación de reglamentos	ADMIN configura reglas y la vista pública refleja las categorías activas
Alta	`MisOrdenes.jsx` forma parte del módulo pero revisar contratos de estados/descargas	Mostrar estados reales, reintentos, PDF de solicitud y carga/descarga protegida del comprobante	Sin `localStorage`; el dueño ve su pago y otro usuario no puede descargarlo
Media	Formulario existente valida nombres/teléfonos y carga datos, pero usa topes fijos	Aplicar errores 409/413/415/422 y límites de servidor, estados vacíos y carga	Mensajes por campo y por servicio sin perder campos ya ingresados
Media	Servicios de listas solicitan `limit=100`	Agregar controles de página y navegación cuando el total exceda la página	No omitir registros fuera de las primeras 100 filas

No se ejecutó compilación/lint frontend para respetar el alcance de no tocar el frontend. Los hallazgos anteriores son inspección estática de archivos, no fallos observados en navegador en esta auditoría.

7. Funcionalidad backend pendiente o limitada

Estas carencias requieren otro cambio backend antes de que el frontend pueda consumirlas; no se deben presentar como rutas ya existentes:

Prioridad	Pendiente backend	Ubicación/decisión necesaria	Criterio de cierre
Alta	Ciclo de vida administrativo completo: borrador, publicar/abrir, cerrar, cancelar, archivar, duplicar plantilla e historial de estado del evento	`Events`, `routers/catalogs.py`, nueva migración y API de lifecycle	Transiciones autorizadas, cancelación conserva historia y duplicado no copia inscripciones/pagos
Alta	Metadatos completos del evento (descripciones, fechas de competencia, modalidad, ubicación, organizador/contacto, visibilidad/material gráfico)	Migración aditiva de `Events`, schemas y contratos públicos/admin	Crear/editar/listar/publicar conserva evento 2026 y soporta nueva edición
Alta	Variaciones de robots por inscripción: hoy `Registrations` tiene un `RobotId` y la clave de duplicidad aplica a un robot	Evaluar tabla `RegistrationRobots` y migración compatible; definir pagos/evaluación por equipo o robot	Un evento que admite varios robots registra y consulta cada robot sin alterar datos anteriores
Media	Categorías son catálogo global con relación evento/categoría, no entidad clonada por edición	Decidir si nombres/descripción/documentos necesitan versiones independientes por evento; hoy reglas y tarifas sí son por evento	Cambiar categoría de otro evento no cambia una edición histórica
Media	Cancelación de eventos y efectos sobre pagos/notificaciones no tiene política ni endpoints	Definir reembolso, correos y estados permitidos antes de codificar	Política aprobada y transiciones probadas con pagos pendientes/aprobados
Media	WebSocket de marcador usa memoria local del proceso	Agregar Redis/pub-sub o limitar deployment a un worker	Clientes conectados a distintos workers reciben el mismo marcador
Media	Prueba con servidor SMTP real no se repitió en esta auditoría; las pruebas automatizadas usan SMTP simulado	Ejecutar prueba controlada de entrega y observar outbox en ambiente autorizado	Email provider acepta el mensaje y equipo verifica Inbox/Spam; nunca incluir credencial en logs

No se debe cancelar, archivar ni duplicar eventos desde la interfaz hasta que las reglas de negocio para pagos y notificaciones estén definidas y exista el API correspondiente.

8. Compatibilidad, seguridad y tratamiento de errores

- No borrar ni recrear la base. Las migraciones 22 y 23 son aditivas y ya se aplicaron a la instancia local `UTA_SUMO_ROBOTICS_CUP`.
- El evento oficial verificado es `UTA SUMO ROBOTICS CUP 2026`; categoría `MINI SUMO AMATEUR RC`; tarifa vigente verificada: USD 15.00; roster oficial: 2 PARTICIPANT, 0 COACH.
- Duplicados de inscripción se protegen por restricción SQL de evento/equipo/robot/categoría. El frontend debe mostrar conflicto y permitir revisar la inscripción existente.
- No exponer `PaymentReference`, correo, teléfono ni comprobantes a roles no autorizados. Ocultar controles en UI mejora experiencia, pero las rutas backend son la autoridad.
- Los errores SQL regresan 503 genérico. No mostrar stack trace o detalle SQL al usuario.
- En carga de archivo manejar 401/403, 404, 409, 413 y 415. No reenviar automáticamente un `POST` de inscripción tras timeout: consultar primero la lista propia para evitar duplicar.
- La paginación máxima habitual es `limit=100`; reportes detallados devuelven `items` y metadatos JSON.

9. Pruebas y resultados reales

Comando ejecutado en `backend/uta_sumo_api`:

```
$env:RUN_SQLSERVER_TESTS='1'
.venv\Scripts\python.exe -m pytest -q
```

Resultado final tras la prueba de reglas configurables: 35 passed. Las pruebas de integración usan SQL Server local y SMTP simulado; verifican auth, permisos, aislamiento, transacciones, inscripciones, pagos, comprobantes, reportes, outbox, scoring y la configuración por evento/categoría. No se probó Google con una ventana OAuth real ni se envió correo real durante esta ejecución.

Migraciones ejecutadas:

```
.venv\Scripts\python.exe scripts\migrate.py
```

Resultado: migraciones previas en `SKIP`, migraciones 22 y 23 aplicadas, sin reset ni eliminación de datos. Se verificó en SQL el evento oficial, la categoría, la tarifa USD 15.00 y las reglas 2 participantes / 0 coaches.

No hay suite frontend detectada en `package.json`; declara `build` y `lint`, no comandos de test. No se ejecutaron porque esta tarea limita los cambios y verificaciones funcionales al backend.

10. Orden de implementación recomendado para frontend

1. Corregir roster dinámico para el evento oficial, leyendo `EventCategories`.
2. Sustituir login/admin mock por JWT y autorización de rutas según rol.
3. Conectar la bandeja administrativa a inscripciones/pagos reales y conservar paginación/filtros.
4. Conectar orden de pago, PDF, comprobante y mensajes de entrega de correo.
5. Conectar reportes paginados y restringir cifras financieras a ADMIN.
6. Reemplazar los resultados en `localStorage` por leaderboard, control de puntuación y WebSocket.
7. Construir vistas de configuración de categorías/reglas/tarifas/reglamentos con APIs disponibles.
8. Coordinar con backend el lifecycle de eventos y soporte de múltiples robots antes de mostrar esos controles.
9. Ejecutar `npm run lint`, `npm run build` y pruebas manuales de cada rol en un ambiente local integrado.

11. Archivos backend/documentación cambiados en esta revisión

- `backend/uta_sumo_api/app/schemas/registration.py`
- `backend/uta_sumo_api/app/schemas/resources.py`
- `backend/uta_sumo_api/main.py`
- `backend/uta_sumo_api/routers/registrations.py`
- `backend/uta_sumo_api/routers/catalogs.py`
- `backend/uta_sumo_api/routers/event_categories.py`
- `backend/uta_sumo_api/scripts/migrate.py`
- `backend/uta_sumo_api/tests/test_foundations.py`
- `backend/uta_sumo_api/tests/test_sqlserver_flows.py`
- `database/scripts/22_event_category_rules.sql`
- `database/scripts/23_official_event_category_rules.sql`
- `backend/uta_sumo_api/README.md`